

Meet Shah, 24116051

This dataset contains P300 recordings from eight healthy participants, captured using a classic 6x6 Donchin and Farewell Speller Matrix. The paradigm utilized a rapid Inter-Stimulus Interval (ISI) of 0.125 ms, which presents a significant decoding challenge due to the high speed. Participants spelled 7 words (5 letters each), with 10 intensification sequences per character. Traditionally, the first 3 words are utilized for model training, while the final 4 words (20 letters total) serve as the test set for word decoding.

```
import numpy as np
import scipy.io as sio
import pandas as pd
import matplotlib.pyplot as plt

# Load the EEG dataset
eeg_content = sio.loadmat("P300S06.mat")

# Display metadata and structure
print(f"File Dictionary Keys: {list(eeg_content.keys())}")
print(f"\nRecorded Channels: {eeg_content['channelNames']}")

# Extract core parameters
sampling_freq = eeg_content["Fs"].item()
raw_record = eeg_content["data"][0, 0]

print(f"\nSignal Matrix:\n{raw_record}")

# Deconstruct the data structure (samples, target, nontarget, trialStart, flashes)
signal_vals, is_target, is_nontarget, start_indices, flash_data = raw_record

# Verify trial distributions
print("\nSummary Statistics:")
print(f"Total Target Count: {np.sum(is_target)}")
print(f"Total Non-Target Count: {np.sum(is_nontarget)}")
```

File Dictionary Keys: ['__header__', '__version__', '__globals__', 'channelRange', 'Fs', 'Trials', 'stims', 'channelIN

Recorded Channels: [[array(['FZ'], dtype='<U2') array(['CZ'], dtype='<U2')
array(['P3'], dtype='<U2') array(['PZ'], dtype='<U2')
array(['P4'], dtype='<U2') array(['P07'], dtype='<U3')
array(['P08'], dtype='<U3') array(['OZ'], dtype='<U2')]]

Signal Matrix:

```
(array([[ -9.27527142,  -0.968431 ,  4.37891197, ..., -7.07288027,
        -0.21369995, -5.13157082],
       [-5.59693527,  1.90235841,  5.57341003, ..., -5.82980061,
        0.70631331, -3.85223413],
       [-0.98746037,  5.13135099,  6.79137564, ..., -3.64892459,
        1.50509155, -2.33834052],
       ...,
       [ 8.22917557, 22.49436951, 20.3970623 , ...,  9.40873051,
        4.8023963 ,  6.1262641 ],
       [ 5.37819386, 20.80926895, 19.26169205, ...,  8.42188549,
        3.47584701,  4.7562499 ],
       [ 4.7656498 , 19.9184227 , 17.82645035, ...,  7.51208782,
        2.48847079,  3.98175883]], array([[0],
       [0],
       ...,
       [0],
       [0],
       [0]], dtype=uint8), array([[0],
       [0],
       ...,
       [0],
       [0],
       [0]], dtype=uint8), array([[ 7465, 17503, 27530, 37554, 47580, 57606, 67630, 77655,
        87682, 97708, 107734, 117762, 127784, 137838, 147820, 157860,
        167886, 177910, 187936, 197959, 207986, 218011, 228035, 238063,
        248093, 258116, 268137, 278175, 288179, 298216, 308240, 318267,
        328292, 338316, 348342]], dtype=int32), array([[ 7465,    31,    12,    1],
       [ 7527,    31,     1,     1],
       [ 7589,    32,    11,     1],
       ...,
       [355675,   31,     4,     1],
       [355735,   29,     8,     1],
```

```
[355735, 31, 2, 1]], dtype=int32))
```

Summary Statistics:
Total Target Count: 140319
Total Non-Target Count: 771702

Filter

```
!pip install mne
```

```
Collecting mne
  Downloading mne-1.11.0-py3-none-any.whl.metadata (15 kB)
Requirement already satisfied: decorator in /usr/local/lib/python3.12/dist-packages (from mne) (4.4.2)
Requirement already satisfied: Jinja2 in /usr/local/lib/python3.12/dist-packages (from mne) (3.1.6)
Requirement already satisfied: lazy-loader>=0.3 in /usr/local/lib/python3.12/dist-packages (from mne) (0.4)
Requirement already satisfied: matplotlib>=3.8 in /usr/local/lib/python3.12/dist-packages (from mne) (3.10.0)
Requirement already satisfied: numpy<3,>=1.26 in /usr/local/lib/python3.12/dist-packages (from mne) (2.0.2)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from mne) (26.0)
Requirement already satisfied: pooch>=1.5 in /usr/local/lib/python3.12/dist-packages (from mne) (1.9.0)
Requirement already satisfied: scipy>=1.11 in /usr/local/lib/python3.12/dist-packages (from mne) (1.16.3)
Requirement already satisfied: tqdm in /usr/local/lib/python3.12/dist-packages (from mne) (4.67.3)
Requirement already satisfied: contourpy>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (0)
Requirement already satisfied: fonttools>=4.22.0 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (4.22.0)
Requirement already satisfied: kiwisolver>=1.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (1.3.1)
Requirement already satisfied: pillow>=8 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (11.3)
Requirement already satisfied: pyparsing>=2.3.1 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (3.1.2)
Requirement already satisfied: python-dateutil>=2.7 in /usr/local/lib/python3.12/dist-packages (from matplotlib>=3.8->mne) (2.9.0)
Requirement already satisfied: platformdirs>=2.5.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.5->mne) (4.2.2)
Requirement already satisfied: requests>=2.19.0 in /usr/local/lib/python3.12/dist-packages (from pooch>=1.5->mne) (2.32.0)
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/python3.12/dist-packages (from Jinja2->mne) (2.1.5)
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12/dist-packages (from python-dateutil>=2.7->matplotlib) (1.16.0)
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->mne) (3.3.0)
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (3.10.1)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (2.2.3)
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests>=2.19.0->pooch) (2025.1.1)
Downloading mne-1.11.0-py3-none-any.whl (7.5 MB)
7.5/7.5 MB 82.4 MB/s eta 0:00:00
Installing collected packages: mne
Successfully installed mne-1.11.0
```

```
import mne

# Define metadata for the MNE Raw object
num_sensors = signal_vals.shape[1]
eeg_info = mne.create_info(
    ch_names=num_sensors,
    sfreq=sampling_freq,
    ch_types='eeg'
)

# Initialize the MNE Raw container (transpose for (n_channels, n_times))
eeg_container = mne.io.RawArray(signal_vals.T, eeg_info)

# Apply sequential frequency filtering
# 1. Notch filter to suppress 50Hz power line interference
# 2. Band-pass filter between 1Hz and 30Hz
eeg_container.notch_filter(50, verbose=False).filter(1, 30, verbose=False)

# Retrieve the processed signals back into the original orientation
samples_filtered = eeg_container.get_data().T
```

```
Creating RawArray with float64 data, n_channels=8, n_times=355872
Range : 0 ... 355871 = 0.000 ... 1423.484 secs
Ready.
```

Visualisations

```
# Force t_start to be a simple integer
t_start = int(start_indices.flatten()[0])

# Determine t_end
if start_indices.size > 1:
    t_end = int(start_indices.flatten()[1])
```

```

else:
    # If only one index exists, default to a 1-second window
    t_end = t_start + int(sampling_freq)

# Calculate the duration in samples
num_samples = t_end - t_start

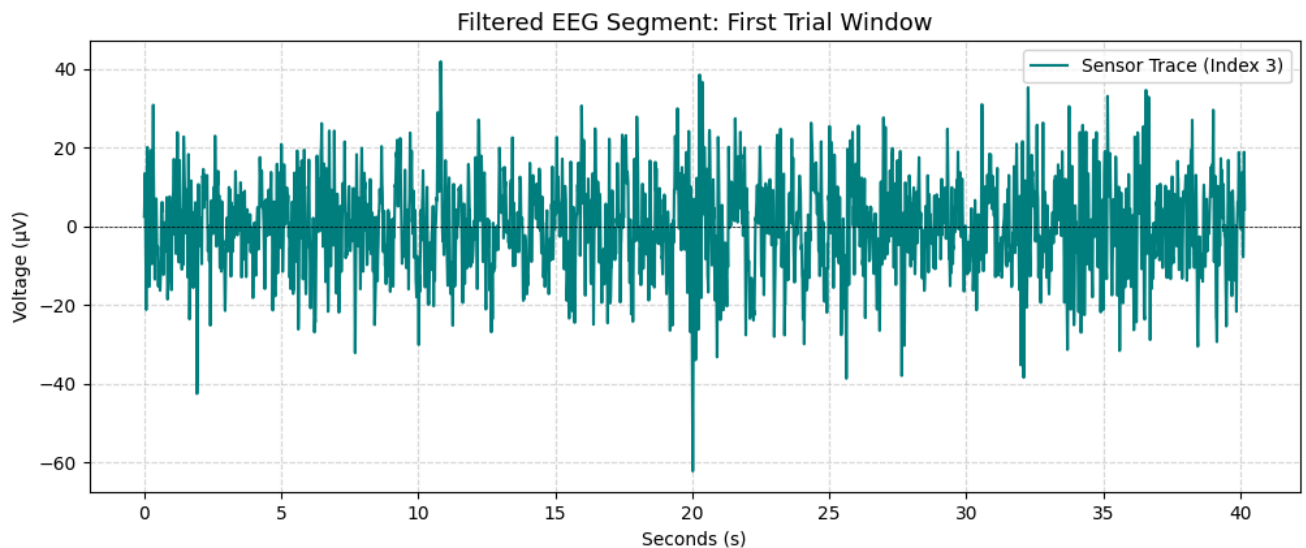
# Create time axis and extract signal
# Flattening sampling_freq
time_scale = np.linspace(0, num_samples / sampling_freq, num_samples)
signal_segment = samples_filtered[t_start:t_end, 3]

# Plotting
plt.figure(figsize=(12, 4.5))
plt.plot(time_scale, signal_segment, color="teal", label="Sensor Trace (Index 3)")

plt.title("Filtered EEG Segment: First Trial Window", fontsize=13)
plt.xlabel("Seconds (s)")
plt.ylabel("Voltage (µV)")
plt.axhline(0, color='black', linewidth=0.5, ls='--') # Add a baseline
plt.grid(True, which='both', linestyle='--', alpha=0.5)
plt.legend()

plt.show()

```



```

epoch_duration = 0.8 # seconds
pts_per_epoch = int(epoch_duration * sampling_freq)
baseline_duration = 0.05 # 50ms for baseline correction

# We use flatten() to ensure we are iterating over a simple list of indices
target_start_indices = [
    int(idx) for idx in start_indices.flatten()
    if is_target.flatten()[int(idx)] == 1
]

erp_epochs = []

for onset in target_start_indices:
    # Define the segment range
    limit = onset + pts_per_epoch

    # Slice the Pz channel (index 3) directly
    if limit <= len(samples_filtered):
        epoch = samples_filtered[onset:limit, 3]

        # baseline correction (subtract mean of the first 50ms)
        baseline_pts = int(baseline_duration * sampling_freq)
        corrected_epoch = epoch - np.mean(epoch[:baseline_pts])

        erp_epochs.append(corrected_epoch)

# Compute the Grand Average
erp_collection = np.array(erp_epochs)

```

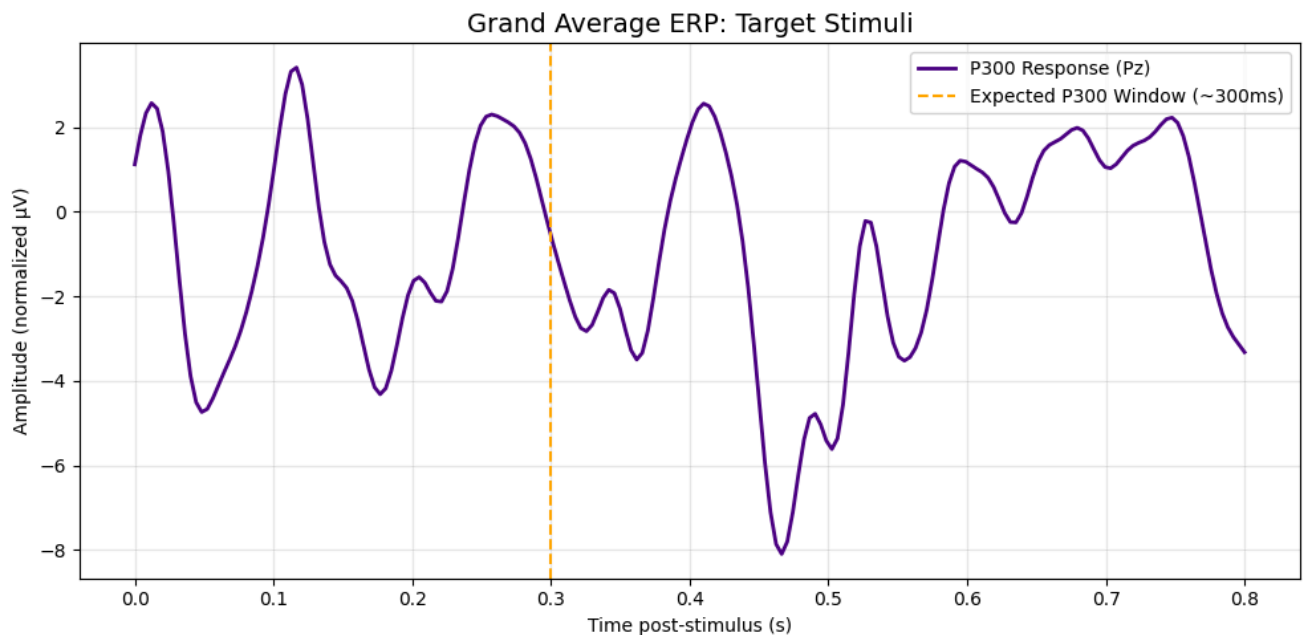
```

grand_average_p300 = np.mean(erp_collection, axis=0)

# Prepare the time vector for the 0.8s window
time_axis_erp = np.linspace(0, epoch_duration, len(grand_average_p300))

# Visualization
plt.figure(figsize=(10, 5))
plt.plot(time_axis_erp, grand_average_p300, color="indigo", linewidth=2, label="P300 Response (Pz)")
plt.title("Grand Average ERP: Target Stimuli", fontsize=14)
plt.xlabel("Time post-stimulus (s)")
plt.ylabel("Amplitude (normalized  $\mu$ V)")
plt.axvline(0.3, color='orange', linestyle='--', label='Expected P300 Window (~300ms)')
plt.grid(True, alpha=0.3)
plt.legend()
plt.tight_layout()
plt.show()

```



- Include the comparison with non-target flashes.

```

import numpy as np
import matplotlib.pyplot as plt

epoch_len = 0.8
n_epoch_samples = int(epoch_len * sampling_freq)
bline_samples = int(0.05 * sampling_freq)

target_set = []
nontarget_set = []

for idx in start_indices.flatten():
    pos = int(idx)
    limit = pos + n_epoch_samples

    if limit > len(samples_filtered):
        continue

    trace = samples_filtered[pos:limit, 3]
    trace = trace - np.mean(trace[:bline_samples])

    if is_target.flatten()[pos] == 1:
        target_set.append(trace)
    else:
        nontarget_set.append(trace)

target_avg = np.mean(target_set, axis=0)
nontarget_avg = np.mean(nontarget_set, axis=0)

timeline = np.linspace(0, epoch_len, target_avg.size)

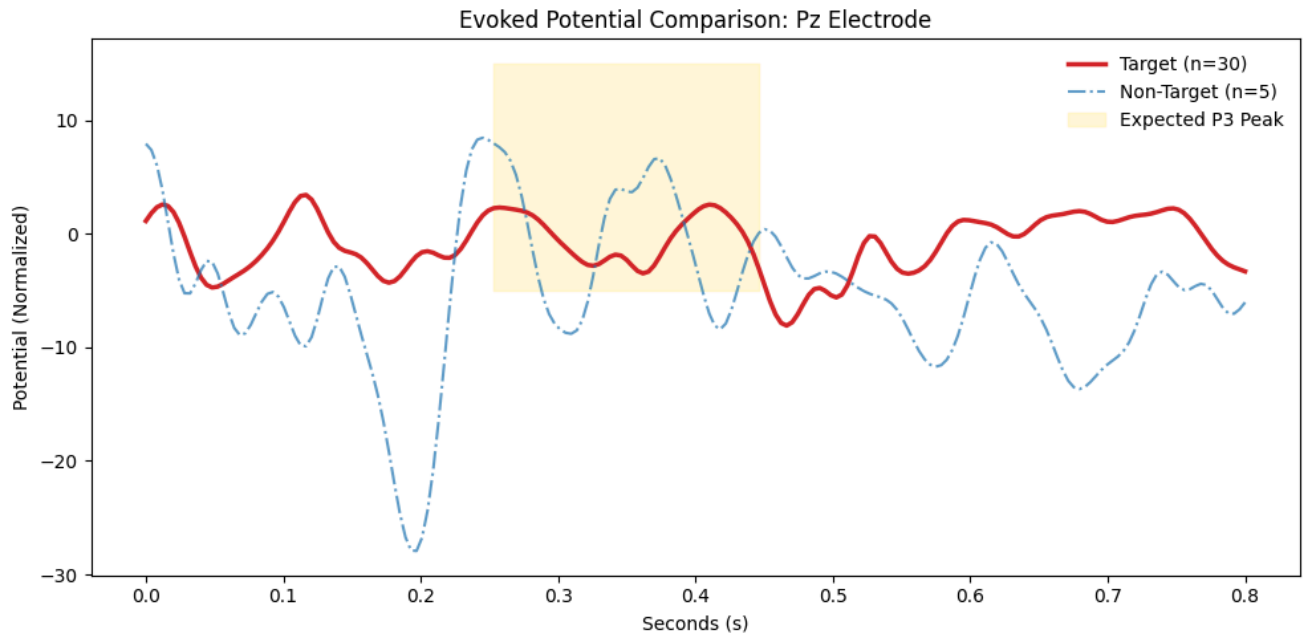
```

```
plt.figure(figsize=(10, 5))

plt.plot(timeline, target_avg, color="#D62728", lw=2.5, label=f"Target (n={len(target_set)})")
plt.plot(timeline, nontarget_avg, color="#1F77B4", lw=1.5, alpha=0.7, ls='-.', label=f"Non-Target (n={len(nontarget_set)})")

plt.fill_between(timeline, -5, 15, where=(timeline >= 0.25) & (timeline <= 0.45), color='gold', alpha=0.15, label="Expected P3 Peak")

plt.xlabel("Seconds (s)")
plt.ylabel("Potential (Normalized)")
plt.title("Evoked Potential Comparison: Pz Electrode")
plt.legend(frameon=False)
plt.tight_layout()
plt.show()
```



The target signal (the red line) shows a small upward bump at about 0.4 seconds, which is the expected brain response we are looking for. However, the non-target signal (the blue line) has a huge, sharp drop around 0.2 seconds that looks like an error or "noise". This likely happened because only 5 trials were averaged for the blue line, which isn't enough to cancel out random background activity like an eye blink or a small movement. Because the blue line is so messy, it makes it much harder to see the actual brain reaction in the red line.

```
import numpy as np
import matplotlib.pyplot as plt

window_sec = 0.8
pts_limit = int(window_sec * sampling_freq)
bline_idx = int(0.05 * sampling_freq)

pos_pool = []
neg_pool = []

for trigger in start_indices.flatten():
    t_0 = int(trigger)
    t_max = t_0 + pts_limit

    if t_max > samples_filtered.shape[0]:
        continue

    # Extracting indices 1 through 4 (Cz, P3, P4, Pz)
    block = samples_filtered[t_0:t_max, 1:5]

    # Vectorized baseline subtraction across the channel axis
    block = block - np.mean(block[:bline_idx, :], axis=0)

    if is_target.flatten()[t_0] == 1:
        pos_pool.append(block)
    else:
        neg_pool.append(block)
```

```

pos_results = np.mean(pos_pool, axis=0)
neg_results = np.mean(neg_pool, axis=0)

x_axis = np.linspace(0, window_sec, pos_results.shape[0])

plt.figure(figsize=(11, 5))

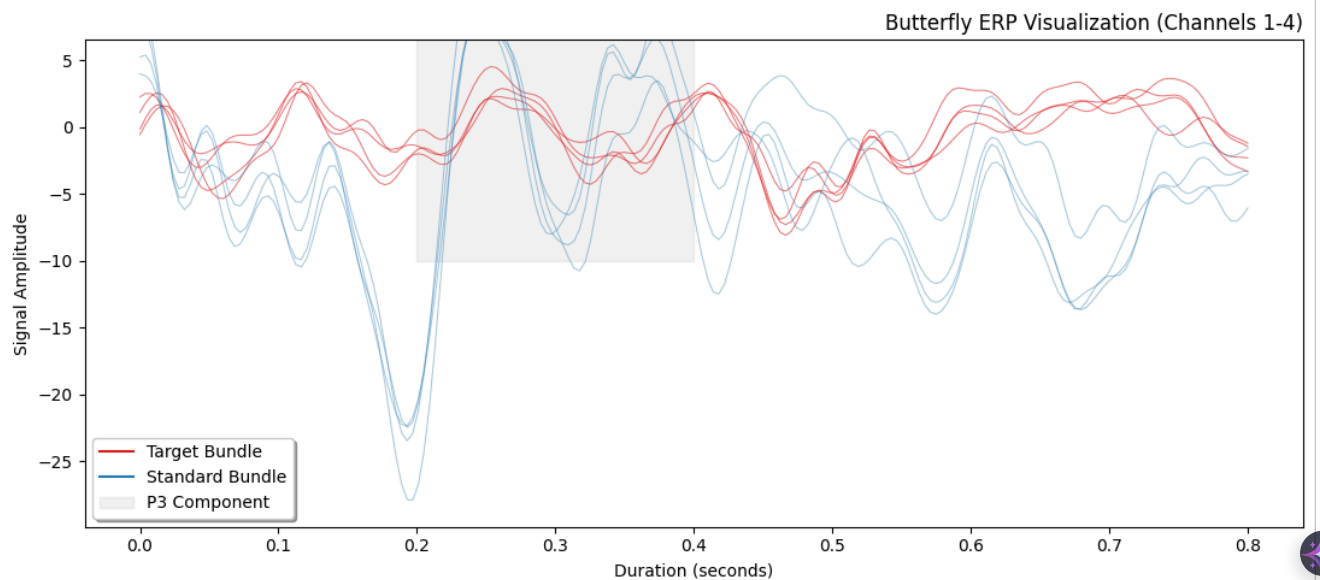
# Plotting each channel from the mean arrays
plt.plot(x_axis, pos_results, color="#E31A1C", lw=0.7, alpha=0.6)
plt.plot(x_axis, neg_results, color="#1F78B4", lw=0.7, alpha=0.4)

# Custom legend proxies
plt.plot([], [], color="#E31A1C", label="Target Bundle")
plt.plot([], [], color="#1F78B4", label="Standard Bundle")

plt.fill_between([0.2, 0.4], -10, 20, color="lightgray", alpha=0.3, label="P3 Component")

plt.title("Butterfly ERP Visualization (Channels 1-4)", loc='right')
plt.xlabel("Duration (seconds)")
plt.ylabel("Signal Amplitude")
plt.ylim(np.min(neg_results)-2, np.max(pos_results)+2)
plt.legend(frameon=True, shadow=True)
plt.tight_layout()
plt.show()

```



In this comparison of several brain channels, the Target Bundle (red lines) shows a consistent, slight upward wave within the shaded P3 window, indicating a unified response to the important signals. However, the Standard Bundle (blue lines) is much more chaotic, with all four channels showing a massive, synchronized "dip" around 0.2 seconds that drops far below the target lines. Because the blue lines are so much noisier and more extreme than the red ones, it suggests that these standard signals haven't been averaged enough to smooth out random background brain "noise," making it harder to clearly distinguish the subtle brain activity we are trying to measure.

```

import mne

window_val = 0.8
n_pts = int(window_val * sampling_freq)
b_pts = int(0.05 * sampling_freq)

target_buffer = []

for trigger_pt in start_indices.flatten():
    idx_start = int(trigger_pt)
    idx_end = idx_start + n_pts

    if idx_end > samples_filtered.shape[0]:
        continue

    segment = samples_filtered[idx_start:idx_end, :]

```

```
segment = segment - segment[:b_pts, :].mean(axis=0)

if is_target.flatten()[idx_start] == 1:
    target_buffer.append(segment)

avg_target_response = np.mean(target_buffer, axis=0)

sensor_labels = ['Fz', 'Cz', 'P3', 'Pz', 'P4', 'P07', 'P08', 'Oz']
eeg_metadata = mne.create_info(ch_names=sensor_labels, sfreq=sampling_freq, ch_types='eeg')

standard_layout = mne.channels.make_standard_montage('standard_1020')
eeg_metadata.set_montage(standard_layout)

# MNE expects Volts, converting from assumed microVolts
erp_evoked = mne.EvokedArray(avg_target_response.T / 1e6, eeg_metadata)

erp_evoked.plot_topomap(
    times=np.linspace(0, 0.75, 16),
    ch_type='eeg',
    ncols=4,
    nrows=4,
    res=128
)
```



